

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет Компьютерного проектирования
Кафедра Инженерной психологии и эргономики
Дисциплина Интерфейсы информационных систем

Лабораторная работа №21
«Построение интерфейса веб-приложения на Vue.js с использованием
экземпляра Vue, компонентов и директив»

Студент группы 310102

(подпись)

Кадол А.П.

Руководитель

(подпись)

Прудник А.М.

Минск 2026

Цель работы: изучить построение интерфейса веб-приложения на *Vue.js* с использованием экземпляра *Vue*, компонентов и директив.

Вариант. Блог-платформа Folio.

Исходные данные. Редакционный блог состоит из шапки, ленты карточек и блока обсуждения. Цвет марки — `#0f3d3a`, фон бумаги — `#faf6f1`, шрифты — *Playfair Display* и *Inter*, радиус карточки — `16 px`, у карточки мягкая тень. В ленте три материала: «Утренние страницы: ритуал, который возвращает голос» Анны Волковой, «Как писать, когда не хочется писать» Петра Лебедева и «Редактура как чтение вслух» Марии Соколовой. Обложки и значки взяты из каталогов `src/assets` и `assets` репозитория и подключены импортом, а не строковым путём.

Теоретические сведения. Корневой экземпляр создаётся вызовом `createApp` и монтируется в элемент `#app`. Интерфейс разбит на однофайловые компоненты с синтаксисом *Options API*. Реактивные поля возвращает функция `data()`. Родитель передаёт данные вниз через `props`: у каждого свойства указаны тип и признак обязательности. Потомок сообщает о действии вызовом `this.$emit`. Список строится директивой `v-for` с уникальным `:key`, ветки интерфейса — парой `v-if` и `v-else`, поле ввода — директивой `v-model`. Сокращение `@click` равносильно `v-on:click`. Стили каждого компонента ограничены блоком `style scoped`. Хранилища *VueX* и *Pinia* не используются: состояние остаётся локальным.

Ход выполнения работы. Сборщик *Vite* подключает `@vitejs/plugin-vue`. Точка входа `src/main.js` создаёт экземпляр и регистрирует корневой компонент `App.vue`. В `data()` корня лежат массив комментариев и счётчик следующего идентификатора. В шаблон комментарии уходят атрибутом `:comments`, а событие `add-comment` обрабатывает метод `addComment`: он добавляет объект с новым `id`, автором «Вы» и переданным текстом. Шапка с логотипом и вводный абзац размечены в том же компоненте.

`ArticleList.vue` хранит в `data()` массив статей и строку `lastReaction`. Тексты берутся из `src/data/articles.js`, обложки импортируются в компонент и подставляются в объекты перед отрисовкой. Директива `v-for` создаёт `ArticleCard` с ключом `article.id` и передаёт одну статью свойством `article`. По событию `like` метод `onLike` записывает заголовок отмеченного материала в `lastReaction`. Пока реакции не было, ветка `v-else` показывает подсказку.

`ArticleCard.vue` объявляет свойство `article` типа `Object` с `required: true`. Счётчик `likes` инициализируется значением `article.likes`, флаг `liked` равен `false`. Кнопка `Like` по `@click` вызывает `onLike`: счётчик увеличивается на единицу, кнопка переходит в отмеченное состояние, наружу уходит `this.$emit("like id)`. Родитель

по этому событию меняет строку над лентой, поэтому событие видно в интерфейсе, а не остаётся пустым вызовом.

`CommentSection.vue` объявляет свойство `comments` типа `Array` с `required: true`. Текст черновика связан с `textarea` через `v-model` и хранится в `data()` вместе с признаком фокуса. Если черновик пуст, `v-if` показывает подсказку и кнопка `Post` недоступна; `v-else` сообщает, что комментарий можно отправить. Метод `submit` отбрасывает пробелы по краям и вызывает `this.$emit("add-comment text")`, после чего очищает поле. Список рисуется при `comments.length` директивой `v-for` с ключом `comment.id`. Если массив пуст, `v-else` выводит фразу «Пока нет комментариев». На собранной странице корень сразу передаёт два комментария, поэтому видна ветка со списком; новый текст появляется в ней после события.

Состав компонентов приведён в таблице 1. Лента до реакции показана на рисунке 1. После нажатия `Like` счётчик первой карточки стал равен 129, что видно на рисунке 2. Отправленный комментарий появился в списке обсуждения, рисунок 3.

Таблица 1 – Компоненты интерфейса блога

Компонент	Вход	Событие наверх	Поля <code>data()</code>
<code>ArticleCard</code>	<code>article</code> , объект, обязателен	<code>like</code>	<code>likes</code> , <code>liked</code>
<code>ArticleList</code>	передаёт статью карточке	слушает <code>like</code>	<code>articles</code> , <code>lastReaction</code>
<code>CommentSection</code>	<code>comments</code> , массив, обязателен	<code>add-comment</code>	<code>draft</code> , <code>focused</code>

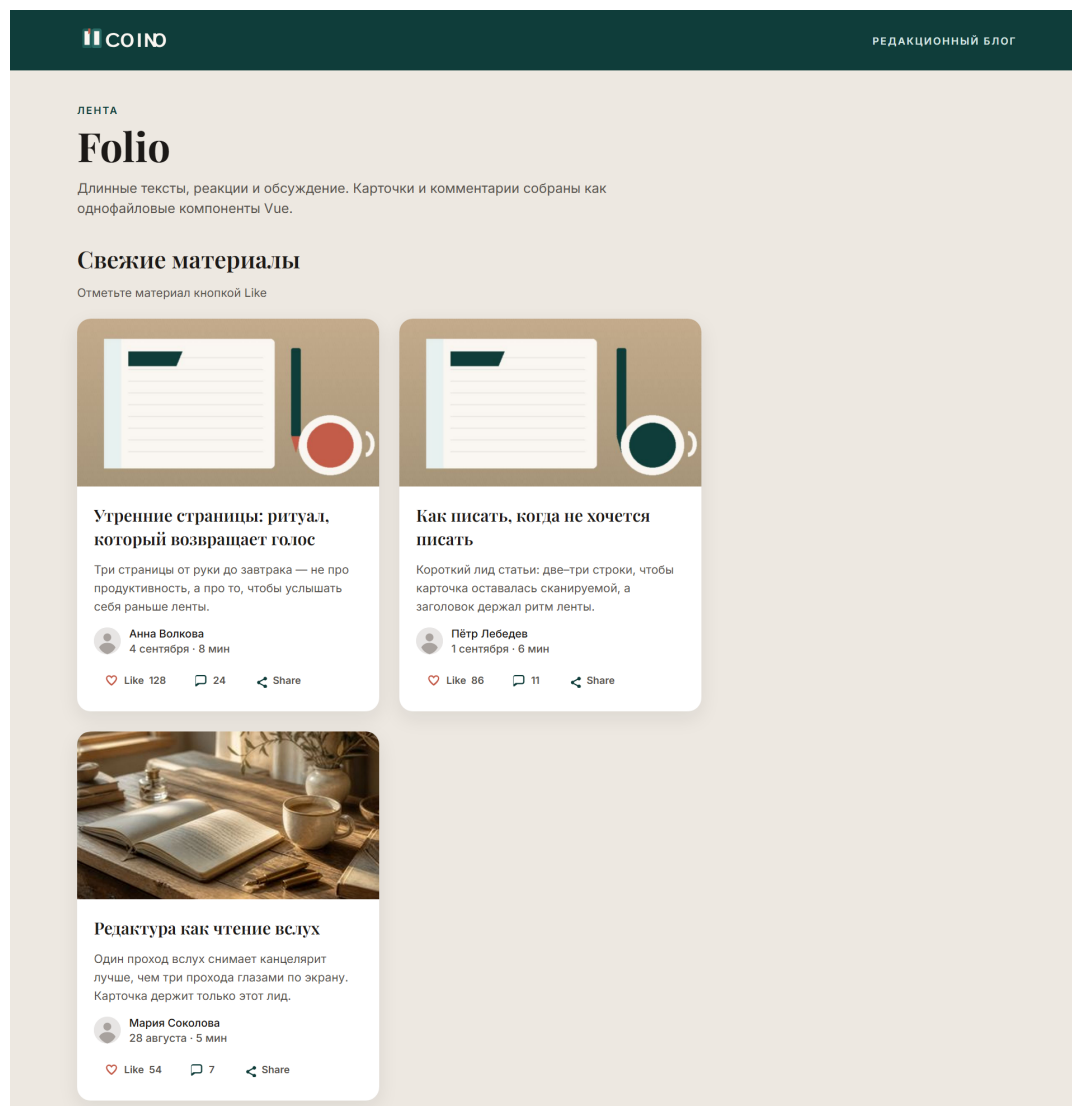
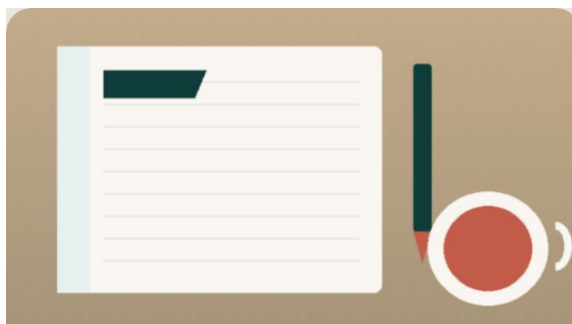


Рисунок 1 – Лента блога: шапка и карточки материалов



Утренние страницы: ритуал, который возвращает голос

Три страницы от руки до завтрака — не про продуктивность, а про то, чтобы услышать себя раньше ленты.



Анна Волкова

4 сентября · 8 мин



Like 129



24



Share

Рисунок 2 — Карточка после нажатия Like: счётчик увеличен до 129

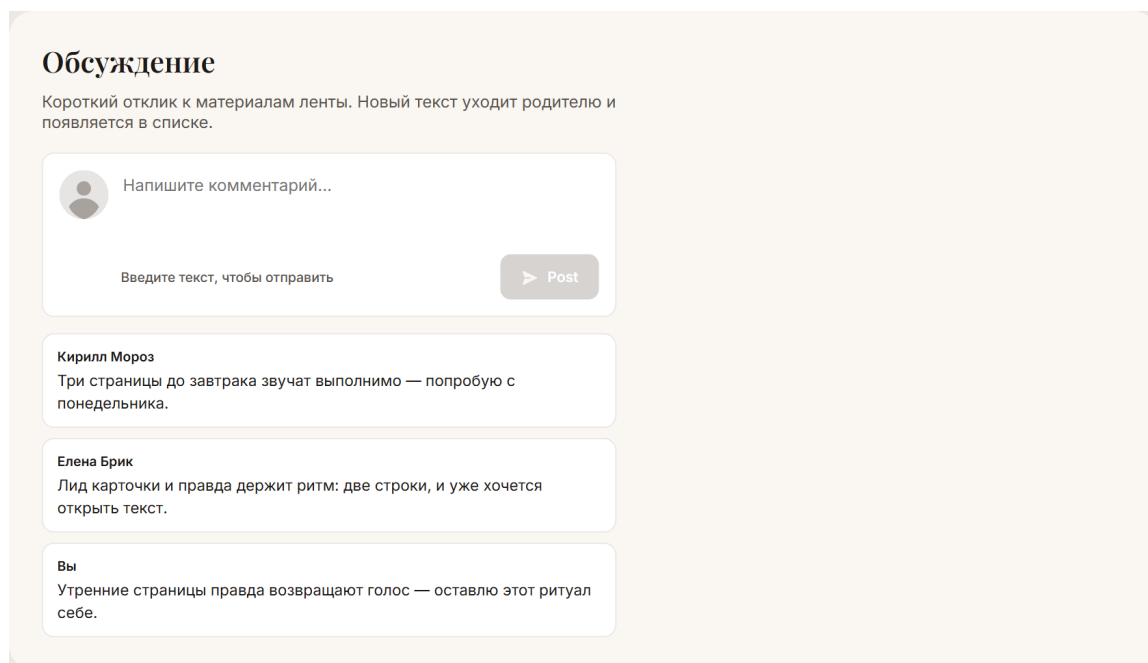


Рисунок 3 — Обсуждение после отправки комментария

Вывод: интерфейс блога собран из корневого экземпляра *Vue* и трёх однофайловых компонентов на *Options API*. Состояние хранится в `data()`, данные идут от родителя к потомку через `props`, действия потомка возвращаются через `$emit`, а списки, пустые ветки и поле комментария заданы директивами `v-for`, `v-if`, `v-else` и `v-model`.